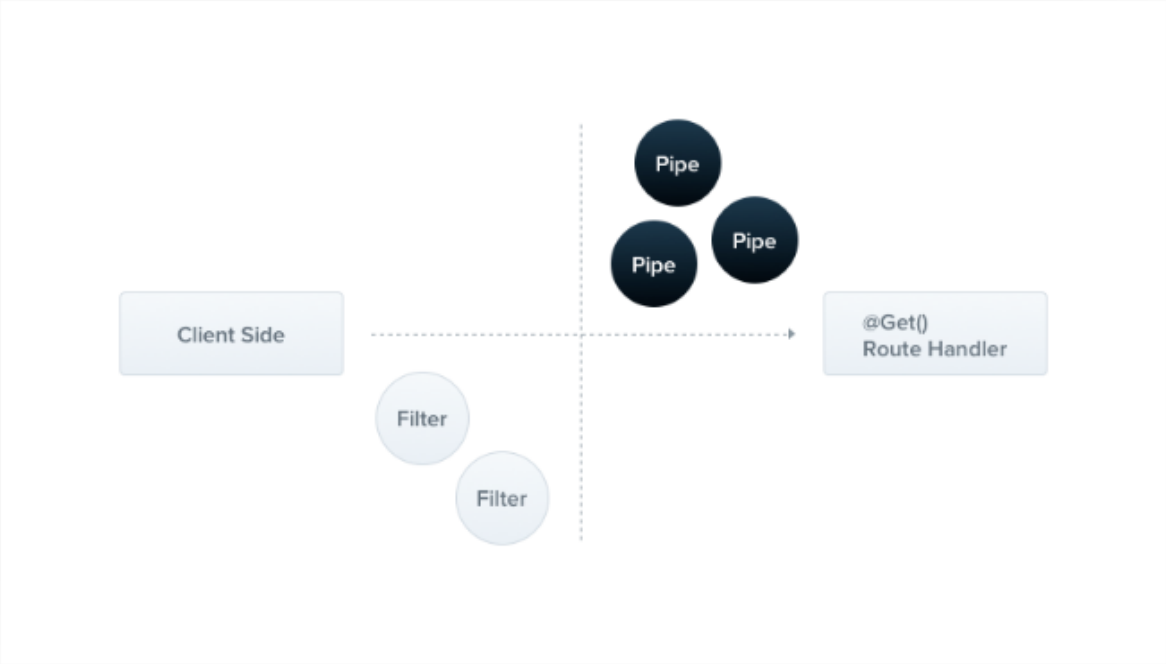
# Pipes

A pipe is a class annotated with the `@Injectable()` decorator, which implements the `PipeTransform` interface.




**********

Pipes have two typical use cases:

- **transformation**: transform input data to the desired form (e.g., from string to integer)
- **validation**: evaluate input data and if valid, simply pass it through unchanged; otherwise, throw an exception

In both cases, pipes operate on the `arguments` being processed by a **controller route handler**. Nest interposes a pipe just before a method is invoked, and the pipe receives the arguments destined for the method and operates on them. Any transformation or validation operation takes place at that time, after which the route handler is invoked with any (potentially) transformed arguments.

Nest comes with a number of built-in pipes that you can use out-of-the-box. You can also build your own custom pipes. In this chapter, we'll introduce the built-in pipes and show how to bind them to route handlers. We'll then examine several custom-built pipes to show how you can build one from scratch.

**********

> **HINT**
> Pipes run inside the exceptions zone. This means that when a Pipe throws an exception it is handled by the exceptions layer (global exceptions filter and any **exceptions filters** that are applied to the current context). Given the above, it should be clear that when an exception is thrown in a Pipe, no controller method is subsequently executed. This gives you a best-practice technique for validating data coming into the application from external sources at the system boundary.

**********

## Built-in pipes

Nest comes with several pipes available out-of-the-box:

- `ValidationPipe`
- `ParseIntPipe`
- `ParseFloatPipe`
- `ParseBoolPipe`
- `ParseArrayPipe`
- `ParseUUIDPipe`
- `ParseEnumPipe`
- `DefaultValuePipe`
- `ParseFilePipe`
- `ParseDatePipe`

They're exported from the `@nestjs/common` package.

**********

## Binding pipes

To use a pipe, we need to bind an instance of the pipe class to the appropriate context. In our `ParseIntPipe` example, we want to associate the pipe with a particular route handler method, and make sure it runs before the method is called. We do so with the following construct, which we'll refer to as binding the pipe at the method parameter level:

```
@Get(':id')
async findOne(@Param('id', ParseIntPipe) id: number) {
  return this.catsService.findOne(id);
}
```

This ensures that one of the following two conditions is true: either the parameter we receive in the `findOne()` method is a number (as expected in our call to `this.catsService.findOne()`), or an exception is thrown before the route handler is called.

**********

```
{
  "statusCode": 400,
  "message": "Validation failed (numeric string is expected)",
  "error": "Bad Request"
}
```

The exception will prevent the body of the `findOne()` method from executing.

In the example above, we pass a class ( `ParseIntPipe` ), not an instance, leaving responsibility for instantiation to the framework and enabling dependency injection. As with pipes and guards, we can instead pass an in-place instance. Passing an in-place instance is useful if we want to customize the built-in pipe's behavior by passing options:

```
@Get(':id')
async findOne(
  @Param('id', new ParseIntPipe({ errorHttpStatusCode: HttpStatus.NOT_ACCEPTABLE }))
  id: number,
) {
  return this.catsService.findOne(id);
}
```

**********

validation.pipe.ts JS

```ts
import { PipeTransform, Injectable, ArgumentMetadata } from '@nestjs/common';

@Injectable()
export class ValidationPipe implements PipeTransform {
  transform(value: any, metadata: ArgumentMetadata) {
    return value;
  }
}
```

**HINT**

`PipeTransform<T, R>` is a generic interface that must be implemented by any pipe. The generic interface uses `T` to indicate the type of the input `value`, and `R` to indicate the return type of the `transform()` method.

**********

The `value` parameter is the currently processed method argument (before it is received by the route handling method), and `metadata` is the currently processed method argument's metadata. The metadata object has these properties:

```ts
export interface ArgumentMetadata {
  type: 'body' | 'query' | 'param' | 'custom';
  metatype?: Type<unknown>;
  data?: string;
}
```

**********

These properties describe the currently processed argument.

| | |
|---|---|
| `type` | Indicates whether the argument is a body `@Body()`, query `@Query()`, param `@Param()`, or a custom parameter (read more here). |
| `metatype` | Provides the metatype of the argument, for example, `String`. Note: the value is `undefined` if you either omit a type declaration in the route handler method signature, or use vanilla JavaScript. |
| `data` | The string passed to the decorator, for example `@Body('string')`. It's `undefined` if you leave the decorator parenthesis empty. |

**********

**WARNING**

TypeScript interfaces disappear during transpilation. Thus, if a method parameter's type is declared as an interface instead of a class, the `metatype` value will be `Object`.

**********

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';
import { CatsFilter } from './cats/cats.filter';
import { ValidationPipe } from '@nestjs/common';
async function bootstrap() {
  const app = await NestFactory.create(AppModule)
  app.enableCors()
  app.useGlobalFilters(new CatsFilter())
  app.useGlobalPipes(new ValidationPipe({
    whitelist: true,
    transform: true,
    forbidNonWhitelisted: true,
  }))
  await app.listen(process.env.PORT ?? 3000)
}
bootstrap();
```

---

```ts
import {IsString, IsInt, IsNotEmpty, Min, Max} from 'class-validator'
export class CreateCatDto {
    @IsString()
    @IsNotEmpty()
    name: string;
    @IsInt()
    @Min(1)
    @Max(50)
    age: number;
    @IsString()
    @IsNotEmpty()
    breed: string;
}
```

**********

> **NOTICE**
> In the case of hybrid apps the `useGlobalPipes()` method doesn't set up pipes for gateways and microservices. For "standard" (non-hybrid) microservice apps, `useGlobalPipes()` does mount pipes globally.

Global pipes are used across the whole application, for every controller and every route handler.

**********

Note that in terms of dependency injection, global pipes registered from outside of any module (with `useGlobalPipes()` as in the example above) cannot inject dependencies since the binding has been done outside the context of any module. In order to solve this issue, you can set up a global pipe **directly from any module** using the following construction:

app.module.ts JS

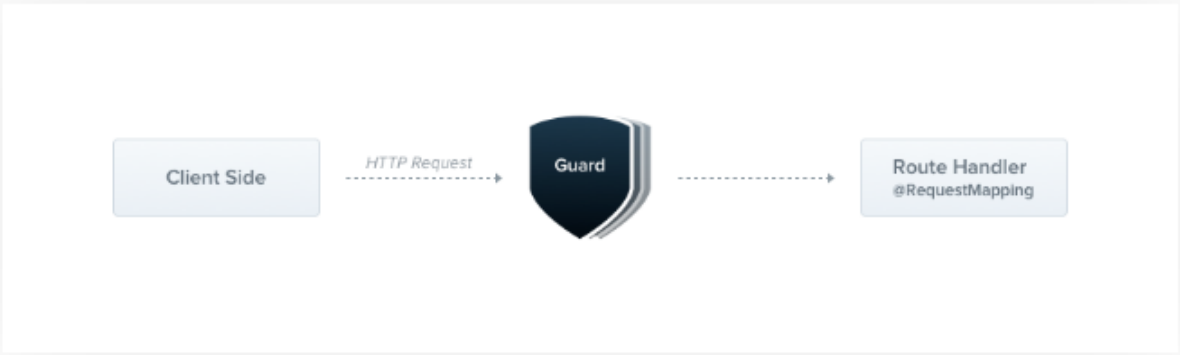
```ts
import { Module } from '@nestjs/common';
import { APP_PIPE } from '@nestjs/core';

@Module({
  providers: [
    {
      provide: APP_PIPE,
      useClass: ValidationPipe,
    },
  ],
})
export class AppModule {}
```

**********

## Guards

A guard is a class annotated with the `@Injectable()` decorator, which implements the `CanActivate` interface.




**********

**Guards** have access to the ExecutionContext instance, and thus know exactly what's going to be executed next. They're designed, much like exception filters, pipes, and interceptors, to let you interpose processing logic at exactly the right point in the request/response cycle, and to do so declaratively. This helps keep your code DRY and declarative.

**********

> **HINT**
> Guards are executed **after** all middleware, but **before** any interceptor or pipe.

**********

To generate guard: *nest g guard AuthGuard*

```
import { CanActivate, ExecutionContext, Injectable } from '@nestjs/common';
import { Request } from 'express';
import { Observable } from 'rxjs';

@Injectable()
export class AuthGuard implements CanActivate {
  canActivate(
    context: ExecutionContext,
  ): boolean | Promise<boolean> | Observable<boolean> {
    const request: Request = context.switchToHttp().getRequest()

    if(request.headers['Auth-Token'] == null) {
      return false
    }else {
      return true
    }
  }
}
```

**********

By extending ArgumentsHost, ExecutionContext also adds several new helper methods that provide additional details about the current execution process. These details can be helpful in building more generic guards that can work across a broad set of controllers, methods, and execution contexts.

**********

**Binding guards**

Like pipes and exception filters, guards can be **controller-scoped**, method-scoped, or global-scoped. Below, we set up a controller-scoped guard using the `@UseGuards()` decorator. This decorator may take a single argument, or a comma-separated list of arguments. This lets you easily apply the appropriate set of guards with one declaration.

JS

```
@Controller('cats')
@UseGuards(RolesGuard)
export class CatsController {}
```

**HINT**
The `@UseGuards()` decorator is imported from the `@nestjs/common` package.

**********

In order to set up a global guard, use the `useGlobalGuards()` method of the Nest application instance:

```
JS

const app = await NestFactory.create(AppModule);
app.useGlobalGuards(new RolesGuard());
```

**NOTICE**
In the case of hybrid apps the `useGlobalGuards()` method doesn't set up guards for gateways and microservices by default (see Hybrid application for information on how to change this behavior). For "standard" (non-hybrid) microservice apps, `useGlobalGuards()` does mount the guards globally.

**********